

INFORME DE PRUEBA DE ESTRES

Plataforma Connect ONION200 / DECODEX Bolivia

Fecha: 6 de mayo de 2026

Ejecutado por: Z.ai - Especialista en Integracion de Redes

Entorno: Z.ai Sandbox (Node.js 24.14.1 / Next.js 16.2.4 / SQLite)

Clasificacion: Confidencial - Uso Interno

1. Resumen Ejecutivo

Se ejecuto una prueba de estres integral sobre la plataforma Connect (ONION200), un sistema Next.js 16 con Prisma ORM y base de datos SQLite. El objetivo fue evaluar la estabilidad del servidor bajo carga concurrente, la integridad de los conectores de base de datos, el flujo de clasificacion de indicadores, y la seguridad general del sistema. La prueba incluyo 6 categorias de test: endpoints concurrentes de lectura, operaciones de escritura, flujos de clasificacion, casos extremos, integridad referencial y analisis de seguridad profundo.

Metrica	Resultado	Estado
Endpoints GET concurrentes (20 req)	200 OK - 154-253ms	APTO
Endpoints POST concurrentes (10 req)	201 Created - 3-18ms	APTO
Escrituras concurrentes en DB (20 req)	201 Created - 162ms total	APTO
Rapid fire 50 requests secuenciales	50/50 OK - 1249ms	APTO
Sync de indicadores (12 indicadores)	8 exitosos, 4 fallidos - 3.9s	PARCIAL
Middleware de autenticacion	DESACTIVADO	CRITICO
Rate limiting en endpoints de escritura	13 endpoints sin proteccion	CRITICO

Tabla 1. Resumen de resultados de la prueba de estres

Se identificaron **6 bug criticos**, **7 vulnerabilidades de seguridad** y **8 inconsistencias de arquitectura**. El sistema muestra un rendimiento sólido en lectura y escritura bajo carga concurrente, pero presenta deficiencias significativas en seguridad (middleware desactivado, sin autenticacion en rutas API), persistencia de datos (cache agresivo en endpoints mutables) y validacion de entradas (endpoints sin Zod ni rate limiting). El servidor se mantuvo estable durante toda la prueba sin consumo excesivo de memoria (239MB RSS, estable).

2. Entorno de Prueba

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Componente' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='SimHeiBold', fontSize=9.5, greek=0,	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Version / Valor' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='SimHeiBold', fontSize=9.5, greek=0, italic=1,
---	--

<code>italic=1, link=[], rise=0, text='Componente', textColor=Color(1,1,1,1), us_lines=[]]</code> 'style': 'bulletText': None 'debug': 0) #Paragraph	<code>link=[], rise=0, text='Version / Valor', textColor=Color(1,1,1,1), us_lines=[]]</code> 'style': 'bulletText': None 'debug': 0) #Paragraph
Sistema Operativo	Linux (Z.ai Sandbox)
Node.js	v24.14.1
Next.js	16.2.4
Prisma ORM	6.19.3
Base de Datos	SQLite (/home/z/my-project/connect/db/custom.db)
CPU	4 nucleos
RAM Disponible	8,082 MB (7,475 MB libre)
Disco	9.9 GB (6.8 GB libre)
Servidor	next start -p 3000 (PID 5364, produccion)
Memory Limit	--max-old-space-size=512

Tabla 2. Configuración del entorno de prueba

El servidor se levanta en modo producción con el script `start.sh` que incluye limitación de memoria y pre-calentamiento de rutas. Se verificó estabilidad previa con 30 consultas rápidas secuenciales (737ms total) antes de iniciar las pruebas de estrés. La base de datos contaba con datos de referencia: 173 Personas, 30 Medios y 12 Ejes Temáticos. Todas las demás tablas estaban vacías al inicio de la prueba.

3. Pruebas de Carga Concurrente

3.1 Lectura Concurrente (GET Endpoints)

Se ejecutaron 20 peticiones concurrentes sobre 6 endpoints principales de lectura. Todos respondieron con HTTP 200. Los tiempos de respuesta se distribuyeron entre 5ms (respuesta en cache) y 128ms (primera petición sin cache). El throughput total fue excelente: 100 peticiones completadas en menos de 260ms en todos los endpoints. La latencia más alta se observó en `/api/indicadores` (128ms max) que realiza consultas adicionales para enriquecer cada indicador con su último valor. Sin embargo, estos tiempos son completamente aceptables para una aplicación de

monitoreo en tiempo real.

<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Endpoint' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='SimHeiBold', fontSize=9.5, greek=0, italic=1, link=[], rise=0, text='Endpoint', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>	<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Registros' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='SimHeiBold', fontSize=9.5, greek=0, italic=1, link=[], rise=0, text='Registros', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>	<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Min' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='SimHeiBold', fontSize=9.5, greek=0, italic=1, link=[], rise=0, text='Min', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>	<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Max' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='SimHeiBold', fontSize=9.5, greek=0, italic=1, link=[], rise=0, text='Max', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>	<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Total (20 req)' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='SimHeiBold', fontSize=9.5, greek=0, italic=1, link=[], rise=0, text='Total (20 req)', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>
/api/personas	173	14ms	65ms	189ms
/api/medios	30	33ms	63ms	197ms
/api/ejes	12	5ms	34ms	160ms
/api/stats	-	9ms	87ms	217ms
/api/indicadores	45	23ms	128ms	253ms
/api/indicadores/historico	-	16ms	35ms	154ms

Tabla 3. Resultados de lectura concurrente (20 peticiones simultaneas)

3.2 Escritura Concurrente (POST Endpoints)

Se crearon 10 clientes y 10 suscriptores gratuitos de forma concurrente. Todos respondieron con HTTP 201. Los tiempos de creacion de clientes oscilaron entre 9ms

y 19ms, mientras que los suscriptores fueron aun mas rapidos (3ms-18ms). Se ejecutaron 20 escrituras concurrentes adicionales de clientes como test de condicion de carrera, completandose todas exitosamente en 162ms. Esto demuestra que la capa de Prisma con SQLite maneja correctamente las escrituras concurrentes sin deadlock ni perdida de datos.

3.3 Rapid Fire (50 peticiones secuenciales)

Se ejecutaron 50 peticiones GET secuenciales a /api/stats sin espera entre ellas. Resultado: 50/50 exitosas en 1,249ms (25ms promedio por peticion). No se observo degradacion de rendimiento ni agotamiento de conexiones. La memoria del servidor se mantuvo estable en 239MB RSS (2.8% del total disponible).

4. Flujo de Clasificacion de Indicadores

El pipeline de captura y clasificacion de indicadores funciona en 4 capas: Scheduler (cron) genera jobs, Check-First verifica accesibilidad, Capture extrae datos de fuentes externas, y Storage+Injection persiste en DB y alimenta boletines. Se probaron los endpoints de sincronizacion individual y general.

4.1 Sincronizacion General (POST /api/indicadores/sync)

El sync general tardo 3.9 segundos y produjo 8 exitosos y 4 fallidos. Los indicadores que fallaron fueron: tc-oficial-bcb (scraping del BCB no disponible), lme-zinc, lme-estano y lme-plomo (fuentes externas no respondieron). Los exitosos incluyeron: lme-cobre (13.230 USD/ton), lme-plata (2,371 USD/ton), y 6 commodities agricolas. Se verifico que los 22 IndicadorValor fueron correctamente persistidos en la base de datos (la sincronizacion anterior tambien contribuyo registros).

4.2 Sincronizacion Individual (POST /api/indicadores/sync/[slug])

	<pre>Paragraph(' caseSensiti ve': 1 'enc oding': ' utf8' 'text' : 'HTTP' '</pre>	<pre>Paragraph(' caseSensitive':</pre>	
--	---	---	--

<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Indicador' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='SimHeiBold', fontSize=9.5, greek=0, italic=1, link=[], rise=0, text='Indicador', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>	<pre>frags': [ParaFrag(__tag__='b', bold=1, fontName='SimHeiBold', fontSize=9.5, greek=0, italic=1, link=[], rise=0, text='HTTP', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>	<pre>1 'encoding': 'utf8' 'text': 'Tiempo' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='SimHei Bold', fontSize=9.5, greek=0, italic=1, link=[], rise=0, text='Tiempo', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>	<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Cache Hit' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='SimHeiBold', fontSize=9.5, greek=0, italic=1, link=[], rise=0, text='Cache Hit', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>
tc-oficial-bcb	200	1,198ms	No (fetch externo)
lme-cobre	200	4ms	Si (cache en memoria)
lme-zinc	200	1,147ms	No (fetch externo)
agr-cafe	200	5ms	Si (cache en memoria)
agr-soya	200	6ms	Si (cache en memoria)

Tabla 4. Sincronizacion individual por indicador

Se observo una diferencia critica entre indicadores con cache y sin cache: los que fueron capturados recientemente responden en menos de 10ms (cache de memoria en el servicio de indicadores con TTL de 1 hora), mientras que los que necesitan fetch externo tardan mas de 1 segundo. Esto es por diseño, pero indica que el sistema de cache esta funcionando correctamente. Los 5 syncs concurrentes sobre el mismo slug (lme-cobre) se completaron en 51ms total sin duplicacion de escrituras problematicas.

5. Bugs e Inconsistencias Encontrados

5.1 Bug Critico: Cache-Control en endpoint mutable

Se detecto que next.config.ts aplica un cache agresivo (s-maxage=300, stale-while-revalidate=600) al endpoint /api/indicadores/historico. Este endpoint retorna datos mutables que cambian despues de cada sincronizacion. El cache causa que las consultas posteriores a un sync retornen datos stale por hasta 5 minutos. Ademias, el historico tiene un bug de parametro: acepta "periodo" pero la documentacion del frontend usa "period", causando que las consultas con "?period=7d" silenciosamente defaulteen a "30d". Finalmente, el capturer-tier1.ts traga errores de escritura en la base de datos silenciosamente, reportando "exitoso" en el sync cuando la persistencia pudo haber fallado.

5.2 Bug Alto: Respuestas de API sin ID en nivel superior

Los endpoints POST de creacion (clientes, suscriptores) envuelven el recurso creado en un objeto anidado: { cliente: { id: "...", ... } }. Los clientes que consumen la API esperan el ID en el nivel superior para operaciones CRUD subsiguientes (read, update, delete). Esto provoca que los flujos de "crear y editar inmediatamente" fallen en el frontend.

5.3 Bug Alto: DELETE devuelve 308 en lugar de 200

El endpoint DELETE /api/clientes/[id]/ con trailing slash retorna HTTP 308 (Permanent Redirect) en lugar de ejecutar la operacion. Next.js App Router con trailingSlash:false (default) redirige URLs con slash final. Los clientes que no siguen redirecciones automaticamente (como curl sin -L) reciben el 308 sin realizar la operacion.

5.4 Bug Alto: /api/search solo acepta POST

El endpoint /api/search solo implementa el handler POST, pero la interfaz de usuario intenta usar GET con query parameter "?q=...". La ausencia del handler GET causa un HTTP 405 Method Not Allowed. Se recomienda agregar un handler GET que lea de searchParams.get("q").

5.5 Bug Medio: /api/jobs POST sin validacion Zod

El endpoint de creacion de jobs usa request.json() con type assertion inseguro (body as {...}) en lugar de guardedParse con schema Zod. Bajo concurrencia, algunas peticiones con body parcialmente vacio reciben 400. Ademias, no tiene rate limiting a diferencia de otros endpoints de escritura, permitiendo inyeccion

masiva de jobs.

5.6 Resumen de Bugs

<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 't ext': '#' frags': [Paragraph(_tag__='b', bold=1, fontName='SimHeiBold', fontSize=9.5, greek=0, italic=1, link=[], rise=0, text='Bug',</pre>	<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Severidad' frags': [Paragraph(__tag__='b', bold=1, fontName='SimHeiBold', fontSize=9.5, greek=0, italic=1, link=[], rise=0, text='Bug',</pre>	<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Archivo' frags': [Paragraph(__tag__='b', bold=1, fontName='SimHeiBold', fontSize=9.5, greek=0, italic=1, link=[], rise=0, text='Archivo',</pre>
--	--	--

ek= 0, it a l ic =1, l i n k= [], r i s e= 0, tex t=' #', t e x tC olo r= C o lo r(1, 1, 1, 1), u s _ li nes =[])] ' sty le' : ' bul le t T ex t': N o n e ' de bu g':	textColor=Color(1,1,1,1), us_lines=[])] 'style': ' bulletText': None 'debug': 0) #Paragraph	italic=1, link=[], rise=0, text='Severi dad', textC o lor=Color(1, 1,1,1), us_lines=[])] 'style': ' bulletText': None 'debu g' : 0) #Parag raph	textColor=Color(1,1,1,1), us_lines=[])] 'style': ' bulletText': None 'debug': 0) #Paragraph
---	---	---	---

0) # P ar a g ra ph			
1	Cache en endpoint mutable (historico)	CRITICO	next.config.ts:49
2	Parametro period vs periodo mismatch	CRITICO	historico/route.ts:18
3	Errores de DB silenciados en capturer	CRITICO	capturer-tier1.ts:840
4	Respuesta POST sin ID en nivel superior	ALTO	clientes/route.ts:123
5	DELETE retorna 308 redirect	ALTO	next.config.ts
6	/api/search sin handler GET	ALTO	search/route.ts
7	Jobs POST sin Zod ni rate limit	MEDIO	jobs/route.ts:48
8	Suscriptores POST response envelope	MEDIO	suscriptores/route.ts:90

Tabla 5. Inventario completo de bugs detectados

6. Analisis de Seguridad

El analisis de seguridad revelo vulnerabilidades criticas. El middleware de autenticacion esta completamente desactivado (archivo renombrado a middleware.ts.disabled). Ninguno de los 49 endpoints API requiere autenticacion, incluyendo endpoints destructivos como /api/admin/purge (que permite borrar todos los datos de clientes y contratos) y /api/seed (que permite reinicializar la base de datos). Los helpers requireAuth() y requireRole() existen en auth-helpers.ts pero no son importados en ningun endpoint.

6.1 Endpoints Protegidos vs Desprotegidos

<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Endpoint' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='SimHeiBold', fontSize=9.5, greek=0, italic=1, link=[], rise=0, text='Endpoint', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>	<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Rate Limit' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='SimHeiBold', fontSize=9.5, greek=0, italic=1, link=[], rise=0, text='Rate Limit', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>	<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Auth' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='SimHeiBold', fontSize=9.5, greek=0, italic=1, link=[], rise=0, text='Auth', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>	<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Riesgo' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='SimHeiBold', fontSize=9.5, greek=0, italic=1, link=[], rise=0, text='Riesgo', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>
--	--	--	--

		bug': 0) #Parag r aph	
/api/admin/purge	No	No	CRITICO
/api/jobs/worker	No	No	CRITICO
/api/seed	Si (2/min)	No	CRITICO
/api/auth/setup	Si (2/min)	No	CRITICO
/api/indicadores/sync	No	No	ALTO
/api/capture	No	No	ALTO
/api/jobs (POST)	No	No	ALTO
/api/clientes (POST)	Si (30/min)	No	MEDIO
/api/menciones	No	No	MEDIO

Tabla 6. Analisis de seguridad por endpoint

6.2 Rate Limiting: Limitaciones Arquitectonicas

El sistema de rate limiting usa un Map in-memory con ventana deslizante (30 req/60s por defecto). Esto funciona correctamente en un entorno de servidor unico, pero presenta dos limitaciones: (1) no persiste entre reinicios del servidor, y (2) no escala horizontalmente en entornos serverless donde multiples instancias comparten la carga. Ademas, la extraccion de IP confia en X-Forwarded-For sin validacion, permitiendo a un cliente spoofear su IP y bypassar el rate limiting. Se recomienda migrar a Upstash Redis para produccion. Actualmente, 13 de 49 endpoints carecen de rate limiting, incluyendo endpoints de alto impacto como sync de indicadores, captura, y el worker de jobs.

6.3 Validacion de Entradas

El sistema cuenta con 17 schemas Zod que cubren las entidades principales (cliente, contrato, persona, suscriptor, entrega, reporte). Sin embargo, los endpoints de jobs usan validacion manual con type assertions inseguras. No existe sanitizacion XSS en los campos de texto (nombre, notas, texto, resumen), lo que podria permitir stored XSS si el contenido se renderiza sin escapar. El campo fotoUrl acepta URLs arbitrarias sin validacion de dominio, lo que podria facilitar ataques SSRF si el servidor realiza fetch a esas URLs.

7. Pruebas de Casos Extremos

<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': ' Caso de Prueba' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='SimHeiBold', fontSize=9.5, greek=0, italic=1, link=[], rise=0, text='Caso de Prueba', textColor=Color(1,1,1,1), us_lines=[])] 'style': ' bulletText': None 'debug': 0) #Paragraph</pre>	<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': ' Resultado' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='SimHeiBold', fontSize=9.5, greek=0, italic=1, link=[], rise=0, text='Resultado', textColor=Color(1,1,1,1), us_lines=[])] 'style': ' bulletText': None 'debug': 0) #Paragraph</pre>	<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'HTTP' ' frags': [ParaFrag(__tag__='b', bold=1, fontName='SimHeiBold', fontSize=9.5, greek=0, italic=1, link=[], rise=0, text='HTTP', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>
ID inexistente (GET /api/clientes/fake)	Mensaje de error claro	404
Body vacio (POST /api/clientes)	Validacion Zod con detalles	400
JSON malformado	"Body debe ser JSON valido"	400
Payload de 10K caracteres	"Too big: max 200 chars"	400
SQL Injection en query param	Respuesta vacia (sin efecto)	200
Ruta inexistente (/api/nonexistent)	404 HTML de Next.js	404
FK invalida (contrato sin cliente)	"Cliente no encontrado"	404
Email duplicado (suscriptor)	"Ya existe un suscriptor..."	400

Tabla 7. Resultados de pruebas de casos extremos

El sistema demostro buena resiliencia ante entradas maliciosas: la inyeccion SQL fue neutralizada por Prisma (que usa consultas parametrizadas), los payloads oversized son truncados por Zod, y las violaciones de restricciones unicas retornan mensajes claros. Sin embargo, el error 404 en rutas inexistentes retorna el HTML por defecto de Next.js en ingles, lo cual deberia personalizarse al espanol para consistencia con la interfaz.

8. Integridad de Base de Datos

Se verifico la integridad referencial completa de la base de datos SQLite. No se encontraron registros huérfanos (Menciones sin Persona o Medio, Contratos sin Cliente). No hay duplicados en slugs de Indicador. Las 20 escrituras concurrentes se completaron sin violación de constraints. El ciclo CRUD completo (Crear, Leer, Actualizar, Eliminar) funciona correctamente para clientes. La paginación se comporta como esperado: las páginas vacías retornan arrays vacíos con el total correcto. Los filtros por departamento, tipo y cámara funcionan correctamente. La base de datos final tras las pruebas contiene: 45 Indicadores, 22 IndicadorValor, 31 Clientes, 11 Suscriptores Gratuitos y 3 Jobs.

9. Recomendaciones (Priorizadas)

<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Prioridad' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='SimHeiBold', fontSize=9.5, greek=0, italic=1, link=[], rise=0, text='Prioridad', textColor=Color(1,1,1,1), us_lines=[])] 'style':</pre>	<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Accion' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='SimHeiBold', fontSize=9.5, greek=0, italic=1, link=[], rise=0, text='Accion', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>	<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Impacto' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='SimHeiBold', fontSize=9.5, greek=0, italic=1, link=[], rise=0, text='Impacto', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>
---	--	--

'bullet Text': None 'de bug': 0) #Pa rAGR aph		
P0	Reactivar middleware.ts (renombrar .disabled)	Bloquea acceso no autorizado a todos los endpoints
P0	Agregar requireAuth() en endpoints de escritura	Protege datos sensibles de clientes y contratos
P0	Eliminar cache-Control en /api/indicadores/historico	Datos en tiempo real despues de sincronizacion
P0	Corregir parametro period vs periodo	Consistencia en la API contract
P1	Agregar ID en nivel superior de respuestas POST	Flujos CRUD completos en el frontend
P1	Agregar handler GET en /api/search	Busqueda funcional desde la UI
P1	Migrar rate limiting a Redis/Upstash	Escalabilidad en produccion multi-instancia
P2	Agregar Zod validation en /api/jobs	Prevencion de inyeccion de jobs invalidos
P2	Sanitizacion XSS en campos de texto	Prevencion de stored XSS
P3	Personalizar pagina 404 al espanol	Consistencia de UX

Tabla 8. Recomendaciones priorizadas

10. Conclusion

La plataforma Connect demuestra un rendimiento tecnico solido en operaciones de lectura y escritura bajo carga concurrente. La arquitectura del pipeline de indicadores con cache en memoria, fallback de fuentes y persistencia en SQLite funciona correctamente. Sin embargo, el sistema presenta deficiencias criticas en tres areas: (1) seguridad, con el middleware desactivado y sin autenticacion en

ningun endpoint; (2) consistencia de datos, con cache agresivo en endpoints mutables y errores de persistencia silenciados; y (3) contratos de API, con parametros inconsistentes y respuestas que no siguen convenciones REST estandar.

Las recomendaciones P0 (reactivar middleware, proteger endpoints, corregir cache y parametros) deben implementarse antes de cualquier despliegue en produccion. Las acciones P1 y P2 pueden abordarse iterativamente. Con estas correcciones, el sistema estara listo para operar 24/7 como requiere la naturaleza del monitoreo de medios e indicadores en tiempo real.